

University of Massachusetts, Amherst
CICS
COMPSCI 683: Artificial Intelligence

Assignment 1: Uninformed Search

Important Instructions:

- Please upload your solution to Gradescope.
 - You may work with a partner on your homework; however, if you choose to do so, **you must** write down the name of your partner on the assignment. In addition, you **may not submit similar/identical solutions**.
 - Provide a mathematical proof/computation steps to all questions, unless explicitly told not to. Solutions with no proof/explanations (even correct ones!) will be awarded no points.
 - Follow the guidelines on writing mathematical proofs provided on Canvas. Remember: we grade your proofs on correctness and clarity, not length. We will deduct points for incorrect claims, even if the proof as a whole is correct. For (a dramatic) example, if you conclude your proof with $1 + 1 = 4$, expect a grade deduction.
 - You may look up combinatorial inequalities and mathematical formulas online, but please do not look up the solutions. Most problems can be found in research papers/online; if you happen upon the solution before figuring it out yourself, mention this in the submission. **Unreferenced copies of online material will be considered plagiarism.**
1. (20 points) A boolean satisfiability problem is given by a set of variables $x_1, \dots, x_n$ (or their negations), organized into *OR clauses*. The clauses are connected by ANDs. This structure is known as Conjunctive Normal Form (CNF).

Consider the following example ($\vee$ stands for logical OR, and $\wedge$ stands for logical AND) that is a CNF formula with two clauses: $C_1 = (x_1 \vee x_3 \vee x_4)$ and $C_2 = (\neg x_1 \vee x_2 \vee \neg x_3)$:

$$\phi(x_1, x_2, x_3, x_4) = C_1 \wedge C_2 = (x_1 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

In order for the formula ϕ to be true, both clauses must be true. In order for C_1 to be true, we must have that $x_1 = \text{true}$, $x_3 = \text{true}$ or $x_4 = \text{true}$; in order for C_2 to be true, we must have that $x_1 = \text{false}$, $x_2 = \text{true}$ or $x_3 = \text{false}$.

A *truth assignment* is a setting of all the variables $x_1, \dots, x_n$ to either `true` or `false`. A *partial* truth assignment is one where some variables *unspecified*, i.e. they do not have a `true` or `false` value. In the example above, setting $x_1 = \text{true}$, $x_2 = \text{false}$ with x_3, x_4 not set is a partial truth assignment.

A truth assignment is *satisfying* if every clause has at least one *literal* (that is, either x_i or $\neg x_i$) that evaluates to `true`. In that case, the formula evaluates to `true`. Our objective is to find a satisfying, complete (i.e. not partial) truth assignment to a general n -variable boolean formula ϕ by searching. We start with all variables unspecified. At each iteration, we select some *unspecified* variable, and set its value to either `true` or `false`.

- (a) (5 points) Give the representation of a state in this problem;
- (b) (5 points) Using the state representation defined above, specify the initial state and goal states.

- (c) (5 points) Define its actions and transition model.
 - (d) (5 points) What is the size of the search space when there are n variables? That is, how many states does it have?
2. (20 points) We are given a search problem where the minimal goal depth is d and the branching factor is a finite value $b \geq 2$. Formally prove that the BFS algorithm terminates in at most $\mathcal{O}(b^d)$ time. We assume that the only operations that take time is exploring a node (not placing it in the frontier), and that each such action takes one time step. You must prove this claim from first principles; in other words, you may not simply say that we have argued this in class and therefore the claim is true.
 3. (20 points) Iterative lengthening search is an iterative analogue of uniform-cost search (UCS). We set an increasing limit ℓ on path cost. Initially, $\ell = 0$. We explore nodes in the same way we would under UCS. At iteration t , if a node is explored whose path cost exceeds the current limit, it is immediately discarded (we add it to a list of discarded nodes D_t). For each new iteration, the limit ℓ is set to the lowest path cost of any node discarded in the previous iteration, i.e., $\ell = \min\{g(n) : n \in D_t\}$, if $D_t \neq \emptyset$, and remains the same otherwise.
 - (a) (6 points) Show that iterative lengthening search is optimal for general path costs. You may assume that all costs are integers (this is not a loss of generality if the search space is finite). You may wish to consider the minimal path cost C ; what happens when we set the limit to be $\ell < C$?
 - (b) (7 points) Consider a uniform tree with branching factor b , solution depth d , and unit step costs (each action costs one unit). How many iterations will iterative lengthening require in the worst case?
 - (c) (7 points) Now consider the case where each step cost is a randomly chosen real number from the interval $[\varepsilon, 1]$ for some $0 < \varepsilon < 1$. How many iterations are required in the worst case? Try to derive the best estimate you can.
 4. (20 points) We have seen various search strategies, and analyzed their worst-case running time. Prove that *any deterministic search algorithm* will, in the worst case, search the entire state space. More formally, we define a search problem by a *finite* set of states $\mathcal{S}$, a set of actions A , and a cost function $c : \mathcal{S} \times A \times \mathcal{S} \rightarrow \{1, \infty\}$ (i.e. the cost is uniform, but some states cannot be reached from some others, or equivalently have a cost of ∞), and a starting state $s_0 \in \mathcal{S}$. We assume that every state $s \in \mathcal{S}$ is reachable from s_0 , i.e. that there is a sequence of actions one can take from state s_0 such that one reaches the state s after performing this sequence of actions, and such that the cost of reaching s is finite. Prove the following theorem.

Theorem 1. *Let Alg be a complete, deterministic uninformed search algorithm. Then for any search problem, there exists some choice of a state $s^* \in \mathcal{S}$ such that Alg checks every state in $\mathcal{S}$.*

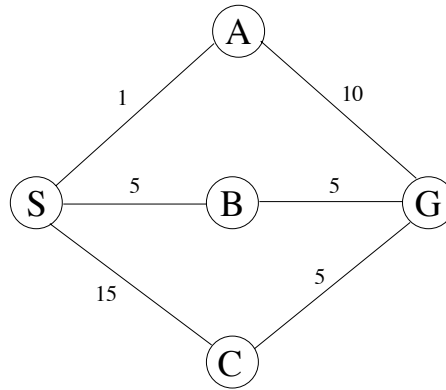


Figure 1: Graph of routes between S and G.

5. (20 points) Consider the graph shown in Figure 1. Let S be the initial state and G be the goal state. The cost of each action is as indicated.
- (a) (8 points) Give a trace of uniform-cost search, assuming we run a graph search; you may implement UCS and submit the program trace.
 - (b) (12 points) When UCS generates G which is the goal with a path cost of 11, why doesn't the algorithm halt and return the search result since the goal has been found? Using this observation, prove the following claim (also shown in class)

Theorem 2. *For any search problem, UCS returns an optimal goal state.*